

ABC454 E - LRUD Moving 题解

题意概括

在一个 $N \times N$ 网格中，从 $(1,1)$ 出发，要求走到 (N,N) ，并且：

- 恰好访问除 (A,B) 以外的所有格子；
- 每个格子最多访问一次。

如果存在这样的路线，就输出一种移动方案。

解题思路

第一步：先判断无解情况

把棋盘黑白染色。

- $(1,1)$ 和 (N,N) 的颜色总是相同；
- 路线会访问 $N^2 - 1$ 个格子，所以路径长度是 $N^2 - 2$ 。

在二分图里：

- 如果路径长度是奇数，起点和终点颜色必须不同；
- 如果路径长度是偶数，起点和终点颜色必须相同。

当 N 为奇数时：

- $N^2 - 2$ 是奇数；
- 但 $(1,1)$ 和 (N,N) 颜色相同。

所以 N 为奇数时一定无解。

当 N 为偶数时：

- 路径长度是偶数，端点颜色相同没有问题；
- 但删去的那个格子必须是另一种颜色，否则剩余两种颜色格子的数量不满足哈密顿路径需要的平衡关系。

因此只有在下面条件同时满足时才可能有解：

- N 是偶数；
- $(A + B)$ 是奇数。

第二步：按两行一组构造

现在只考虑 N 为偶数且 $(A + B)$ 为奇数的情况。

把行按两行一组分成：

- $(1, 2)$
- $(3, 4)$
- ...

设包含被删格子的那一组行为特殊组。由于 $(A + B)$ 为奇数，这个被删格子在特殊组中一定是：

- 上面一行的偶数列，或
- 下面一行的奇数列。

特殊组上方

对于特殊组上方的每一组两行，直接用标准蛇形走法：

- 先把上面一行从左走到右；
- 下去一格；
- 再把下面一行从右走到左；
- 如果下面还有行，就再下去一格进入下一组。

这样会一直走到特殊组左上角。

特殊组内部

特殊组只有两行，可以分类讨论：

1. 被删格子在上面一行的偶数列：
 - 在它左边的完整两列块中反复走 DRUR
 - 到它前一系列后走 DR
 - 在它右边反复走 RURD
2. 被删格子在下面一行的奇数列：
 - 在它左边的完整两列块中反复走 DRUR
 - 到它这一列后走 RD
 - 在它右边反复走 RURD

这样可以刚好绕开被删格子，并从特殊组右下角走出。

特殊组下方

对下面剩余的每一组两行，从右边开始做对称蛇形：

- 先下去进入这一组；
- 把上面一行从右走到左；

- 下去一格;
- 再把下面一行从左走到右。

最终会停在 (N, N) 。

正确性说明

先看判定部分。

当 N 为奇数时, 路径长度 $N^2 - 2$ 为奇数, 而二分图中奇数长度路径的两个端点必须颜色不同, 但 $(1, 1)$ 和 (N, N) 颜色相同, 所以一定无解。

当 N 为偶数时, 剩余格子总数是奇数。要让一条从同色端点出发并结束的简单路径覆盖全部剩余格子, 两种颜色的格子数必须相差 1, 而且多出来的必须是端点所在颜色。因此被删掉的格子必须是另一种颜色, 也就是 $(A + B)$ 必须是奇数。

再看构造部分。

- 特殊组上方的每个两行块都被标准蛇形完整覆盖, 且块与块之间首尾相接;
- 特殊组内部的构造在两行范围内恰好访问除 (A, B) 外的所有格子, 并从左侧入口连到右侧出口;
- 特殊组下方的每个两行块也被完整覆盖, 并且和上一步的出口自然相接。

整条路线:

- 从 $(1, 1)$ 出发;
- 不会重复访问任何格子;
- 恰好漏掉 (A, B) ;
- 最终到达 (N, N) 。

因此构造正确。

复杂度

每个格子只对应常数输出。

- 时间复杂度: $O(N^2)$
- 空间复杂度: $O(N^2)$ (计入输出字符串)

参考实现

```
#include<bits/stdc++.h>
using namespace std;

int main(){
```

```
int T;
cin >> T;

while(T--){
    int n, a, b;
    cin >> n >> a >> b;

    if(n % 2 == 1 || (a + b) % 2 == 0){
        cout << "No\n";
        continue;
    }

    string ans;
    int special = (a % 2 == 1 ? a : a - 1);

    // 特殊组上方的两行块，从左边蛇形向下走
    for(int r=1; r<special; r+=2){
        ans.append(n - 1, 'R');
        ans.push_back('D');
        ans.append(n - 1, 'L');
        ans.push_back('D');
    }

    // 特殊组内部，绕开被删掉的格子
    if(a == special){
        for(int c=1; c<=b - 2; c+=2){
            ans += "DRUR";
        }
        ans += "DR";
        for(int c=b + 1; c<=n - 1; c+=2){
            ans += "RURD";
        }
    } else {
        for(int c=1; c<=b - 2; c+=2){
            ans += "DRUR";
        }
        ans += "RD";
        for(int c=b + 2; c<=n - 1; c+=2){
            ans += "RURD";
        }
    }
}
```

```
}  
  
// 特殊组下方的两行块，从右边蛇形向下走  
for(int r=special + 2; r<=n - 1; r+=2){  
    ans.push_back('D');  
    ans.append(n - 1, 'L');  
    ans.push_back('D');  
    ans.append(n - 1, 'R');  
}  
  
cout << "Yes\n";  
cout << ans << '\n';  
}  
  
return 0;  
}
```